

DUOC UC

INFORME INDIVIDUAL

Proyecto

Sistema de Reserva de Canchas Deportivas

Alumno: Pablo Acuña

Profesor: Carlos Martínez

Asignatura: Desarrollo Full Stack I

Fecha: Julio 2026

Santiago, Chile

1. Introducción

El proyecto consistió en desarrollar el backend de un sistema de reserva de canchas deportivas utilizando una arquitectura basada en microservicios con Spring Boot. La idea era que cada microservicio se encargara de una función específica del sistema y que todos pudieran comunicarse entre sí para completar el proceso de una reserva.

Mi participación comenzó desde la primera etapa del proyecto, donde trabajé en la implementación de algunos de los primeros microservicios. A medida que el proyecto fue avanzando también me fui involucrando en otras tareas, como integrar los distintos servicios, agregar nuevas funcionalidades, corregir problemas que iban apareciendo durante las pruebas y preparar el sistema para poder desplegarlo en Render.

En la última etapa también me encargué de revisar que todos los microservicios funcionaran correctamente entre sí, actualizar la documentación del proyecto y dejar el repositorio listo para la entrega final.

En este informe explicaré los principales trabajos que realicé durante el semestre, las dificultades que fui enfrentando y cómo las resolví hasta dejar el proyecto funcionando tanto de forma local como en la nube.

2. Descripción del proyecto

Cuando comenzó el proyecto, el objetivo era desarrollar un sistema de reserva de canchas deportivas utilizando una arquitectura basada en microservicios. Cada evaluación fue incorporando nuevas funcionalidades, por lo que el sistema fue creciendo hasta contar con servicios para administrar usuarios, recintos, canchas, horarios, reservas, pagos, precios, notificaciones, reseñas y mantenimientos.

Al finalizar el desarrollo, todos los microservicios quedaron integrados, permitiendo realizar el flujo completo de una reserva y dejando el sistema preparado para ejecutarse tanto de forma local como en la nube.

3. Desarrollo realizado

3.1 Desarrollo de los primeros microservicios

Mi trabajo comenzó desde las primeras semanas del proyecto con el desarrollo del **User Service**, donde implementé las funcionalidades principales para la administración de usuarios. Una vez terminado ese microservicio continué con el desarrollo de **Recinto Service**, **Cancha Service** y **Horario Service**, agregando en cada uno las funcionalidades que se iban solicitando en las distintas evaluaciones.

En esta etapa el objetivo fue dejar cada microservicio funcionando de manera independiente, implementando sus operaciones CRUD, validaciones, manejo de excepciones y las pruebas necesarias para comprobar que todo funcionara correctamente antes de comenzar la integración con el resto del sistema.

3.2 Mejoras realizadas durante el semestre

Una vez que los primeros microservicios estuvieron funcionando, el trabajo pasó a enfocarse en ir incorporando todas las mejoras que se fueron solicitando durante el semestre.

Entre las principales tareas que realicé estuvieron la incorporación de **Spring Security**, la configuración de **Flyway** para controlar las migraciones de la base de datos, la implementación de **Swagger/OpenAPI** para documentar los servicios, la creación de **pruebas unitarias**, el manejo personalizado de errores y la incorporación de **Docker** para facilitar la ejecución del proyecto.

A medida que avanzaban las evaluaciones también fui actualizando los microservicios existentes para incorporar estas mejoras, ya que muchas de ellas no solo debían implementarse en un servicio nuevo, sino también adaptarse a los que ya estaban desarrollados.

3.3 Integración de la arquitectura

Cuando los microservicios ya estaban desarrollados, comencé a trabajar en la integración del proyecto completo. En esta etapa implementé **API Gateway** para tener un único punto de entrada a todos los servicios y luego incorporé **Eureka Discovery Server**, permitiendo que los microservicios se encontraran automáticamente sin depender de direcciones configuradas manualmente.

También adapté la comunicación entre los distintos servicios utilizando **RestClient** junto con **Spring Cloud LoadBalancer**, realizando varias correcciones para que las llamadas entre microservicios funcionaran correctamente. Durante este proceso fui revisando distintos errores que aparecían al integrar los servicios y realizando las modificaciones necesarias hasta lograr que todo el sistema pudiera comunicarse de forma estable.

Además, configuré **Docker Compose** para poder levantar toda la arquitectura de manera automática, facilitando las pruebas y evitando tener que iniciar cada microservicio por separado.

Además, configuré Swagger para documentar los distintos microservicios y posteriormente centralicé toda la documentación a través del API Gateway, permitiendo acceder a las APIs desde un único punto.

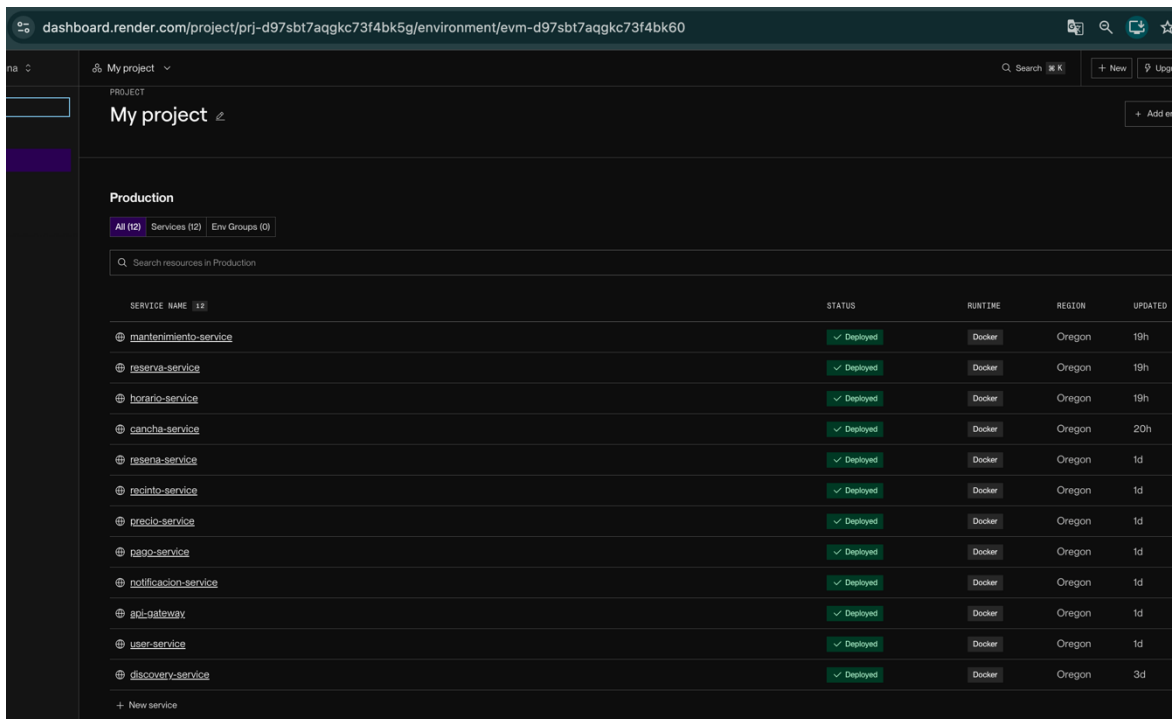
← → ↺ discovery-service-jp0a.onrender.com		Lease expiration enabled	false
		Renews threshold	20
		Renews (last min)	20
EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT. RENEWALS ARE LESSER THAN THRESHOLD AND HENCE THE INSTANCES ARE NOT SAFE.			
DS Replicas			
localhost			
Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (1)	(1)	UP (1) - srv-d98kt23eo5us73dmdslg-hibernate-78dc745665-scmqs:api-gateway:10000
CANCHA-SERVICE	n/a (1)	(1)	UP (1) - srv-d98lwl1o3t8c739s1uc0-hibernate-574bc7bf4d-x8hgq:cancha-service:10000
HORARIO-SERVICE	n/a (1)	(1)	UP (1) - srv-d98jvctqb8s73fos0dg-hibernate-ff9596c64-4t7wc:horario-service:10000
MANTENIMIENTO-SERVICE	n/a (1)	(1)	UP (1) - srv-d98qbnho3t8c73eaoccg-hibernate-84b88cdfc-8jz79:mantenimiento-service:10000
NOTIFICACION-SERVICE	n/a (1)	(1)	UP (1) - srv-d98kkol7vvec739q5k60-hibernate-d5644d9c7-7h2gc:notificacion-service:10000
PAGO-SERVICE	n/a (1)	(1)	UP (1) - srv-d98kfemtrd3s73ercvn0-hibernate-6f6d4ff64-xhqr:pago-service:10000
PRECIO-SERVICE	n/a (1)	(1)	UP (1) - srv-d98kbfavr4c739ma5ug-hibernate-9b5ffcf4f8-nctvf:precio-service:10000
RECINTO-SERVICE	n/a (1)	(1)	UP (1) - srv-d98lap5aeets7381d7g0-hibernate-57898db95-lnr95:recinto-service:10000
RESENA-SERVICE	n/a (1)	(1)	UP (1) - srv-d98koge1p3s738kit10-hibernate-57cf9d64d7-cc75b:resena-service:10000
RESERVA-SERVICE	n/a (1)	(1)	UP (1) - srv-d98k6knayr4c739m2erg-hibernate-86b86b47d-mqx2h:reserva-service:10000
USER-SERVICE	n/a (1)	(1)	UP (1) - srv-d98hc3ernols73fatu0g-hibernate-5fdd6f65b-td7h5:user-service:10000
General Info			

3.4 Preparación del despliegue

Para esto adapté prácticamente todos los microservicios para que pudieran ejecutarse en **Render**, utilizando una base de datos **MySQL alojada en Railway**. Además, fui revisando cada microservicio de manera individual para adaptar las configuraciones que eran necesarias antes del despliegue definitivo. Esto implicó modificar configuraciones, variables de entorno, puertos y conexiones para que todos los servicios pudieran comunicarse correctamente una vez publicados.

Durante esta etapa también corregí distintos problemas que aparecieron al realizar las pruebas, como configuraciones de **Swagger**, enlaces del **API Gateway**, configuración de **LoadBalancer**, comunicación entre servicios y algunos errores específicos que impedían que ciertos microservicios iniciaran correctamente.

Como apoyo al despliegue, también preparé scripts para facilitar el levantamiento de los servicios y fui realizando pruebas hasta comprobar que toda la arquitectura funcionara correctamente tanto en el entorno local como en Render.



The screenshot shows the Render dashboard for a project named "My project". The "Production" environment is selected, showing a list of 12 services. All services are in a "Deployed" status, running on Docker in the Oregon region. The services listed are: mantenimiento-service, reserva-service, horario-service, cancha-service, reserva-service, recinto-service, precio-service, pago-service, notificacion-service, api-gateway, user-service, and discovery-service. The "discovery-service" was updated 3 days ago, while the others were updated 19 hours to 1 day ago.

SERVICE NAME	STATUS	RUNTIME	REGION	UPDATED
mantenimiento-service	✓ Deployed	Docker	Oregon	19h
reserva-service	✓ Deployed	Docker	Oregon	19h
horario-service	✓ Deployed	Docker	Oregon	19h
cancha-service	✓ Deployed	Docker	Oregon	20h
reserva-service	✓ Deployed	Docker	Oregon	1d
recinto-service	✓ Deployed	Docker	Oregon	1d
precio-service	✓ Deployed	Docker	Oregon	1d
pago-service	✓ Deployed	Docker	Oregon	1d
notificacion-service	✓ Deployed	Docker	Oregon	1d
api-gateway	✓ Deployed	Docker	Oregon	1d
user-service	✓ Deployed	Docker	Oregon	1d
discovery-service	✓ Deployed	Docker	Oregon	3d

3.5 Correcciones finales y documentación

En la última etapa del proyecto me enfoqué en dejar el repositorio ordenado y listo para la entrega. Actualicé completamente el **README**, incorporando una descripción más completa del proyecto, la arquitectura utilizada, las instrucciones para ejecutar el sistema y la información necesaria para que cualquier persona pudiera levantar la solución.

Además, elaboré el **Manual Técnico** y el **Manual de Usuario**, con el objetivo de dejar documentado el funcionamiento general del sistema y facilitar su utilización.

También realicé una limpieza del repositorio eliminando archivos temporales que no era necesario versionar, como algunos archivos generados dentro de las carpetas **target**, dejando el proyecto mucho más limpio y fácil de mantener.

Para evitar afectar el trabajo principal durante todo el desarrollo, cada conjunto importante de cambios fue realizado en una **rama de trabajo**, donde iba probando las modificaciones antes de integrarlas al proyecto. Una vez verificadas las pruebas, los cambios fueron fusionándose con la rama principal de desarrollo, dejando una versión estable para la entrega final.

4. Evidencia de mi participación

Durante el desarrollo del proyecto fui realizando cambios de forma continua durante todo el semestre. Para evitar afectar el trabajo principal, cada modificación importante fue desarrollada en una rama de trabajo y, una vez validada, fue integrada al proyecto. Mi participación puede comprobarse a través del historial de commits del repositorio, donde se evidencia el desarrollo de microservicios, mejoras de arquitectura, despliegue, documentación y correcciones finales.

Etapa	Principales aportes	Commits de respaldo
Desarrollo inicial	Creación de User Service, Recinto Service, Cancha Service y Horario Service.	18b66e1, f957fd3, bf1eb1e, 487cc92
Mejoras de los microservicios	Spring Security, Flyway, validaciones, manejo de errores, pruebas unitarias y Docker.	9677e79, 342998b, 74542aa
Arquitectura	Implementación de API Gateway, Eureka Discovery Server, Swagger centralizado, Docker Compose y comunicación entre microservicios mediante RestClient y LoadBalancer.	9ffa601, d0fcd14, b6dade5, ff51137, 0a0d855
Despliegue	Adaptación de los microservicios para Render, utilización de Railway como base de datos MySQL, configuración de variables de entorno y corrección de errores de despliegue.	2eb9311, d114f64, 4136bc3, 3ed45e0, 8720e13, 34307d5
Ajustes finales	Correcciones de Swagger, mantenimiento-service, API Gateway, RestClient, LoadBalancer y configuraciones finales.	4c15e8a, 3b0515e, 5175300, 9f56e6c, 16b7678, 88f9f39
Documentación	Actualización del README, elaboración del Manual Técnico y Manual de Usuario, limpieza del repositorio y eliminación de archivos innecesarios.	a43523b, 8e94a8f, cdafcc7

5. Archivos en los que trabajé

Durante el semestre trabajé principalmente en los siguientes componentes del proyecto:

- User Service.
- Recinto Service.
- Cancha Service.
- Horario Service.
- API Gateway.
- Eureka Discovery Server.
- Configuración de Docker y Docker Compose.
- Configuración de Swagger/OpenAPI.
- Configuración de Spring Security.
- RestClient y comunicación entre microservicios.
- Configuración para despliegue en Render.
- Configuración de la base de datos MySQL en Railway.
- README del proyecto.
- Manual Técnico.
- Manual de Usuario.

5.1 Pruebas realizadas

Como parte del desarrollo también implementé pruebas unitarias en los microservicios donde eran requeridas por la evaluación, utilizando **JUnit 5** y **Mockito**. Estas pruebas permitieron verificar el funcionamiento de los servicios, validar la lógica implementada y comprobar distintos escenarios antes de integrar los cambios al resto del proyecto.

Además de las pruebas unitarias, fui realizando pruebas REST mediante **Swagger/OpenAPI** para comprobar que los endpoints respondieran correctamente y que la comunicación entre los microservicios funcionara como se esperaba. Estas validaciones fueron especialmente importantes durante la integración del API Gateway, Eureka Discovery Server y el despliegue final en Render.

Gracias a estas pruebas fue posible detectar distintos errores durante el desarrollo y corregirlos antes de realizar la entrega final.

6. Aprendizajes

Durante el desarrollo del proyecto fui aprendiendo distintas cosas a medida que avanzaban las evaluaciones. Al principio mi trabajo estuvo más enfocado en desarrollar microservicios, pero después también me tocó integrar todos los servicios, revisar configuraciones y solucionar los problemas que iban apareciendo durante las pruebas.

Otro aprendizaje importante fue trabajar utilizando ramas en Git. Esto me permitió ir realizando cambios de forma más ordenada, probar nuevas implementaciones sin afectar el proyecto principal y mantener un mejor control de cada modificación que iba realizando.

También aprendí cómo preparar un proyecto para un despliegue en la nube utilizando Render y una base de datos en Railway. Antes pensaba que el trabajo terminaba cuando el proyecto funcionaba de forma local, pero durante esta etapa entendí que todavía quedaban varias configuraciones y pruebas antes de dejar el sistema completamente operativo.

7. Conclusión

Este proyecto me permitió participar en distintas etapas del desarrollo, desde la creación de los primeros microservicios hasta la integración del proyecto completo, el despliegue en la nube y la preparación de la documentación final.

En cada evaluación fui incorporando nuevas funcionalidades y realizando mejoras sobre el trabajo ya desarrollado, lo que me permitió entender mejor cómo va evolucionando un proyecto durante el semestre. Al finalizar, pude dejar el sistema funcionando, con la documentación actualizada y preparado para su entrega.